07-structuring-code-examples/01/getting_the_words_right.rb

```
RSpec.describe 'Deprecations' do
  specify 'MyGem.config is deprecated in favor of MyGem.configure'
  specify 'MyGem.run is deprecated in favor of MyGem.start'
end
```

We've grouped these examples along a cross-cutting concern—library deprecations—instead of a common subject. specify works well for cases like these, where each example has its own subject.

Defining Your Own Names

You're not limited to these aliases. RSpec allows you to define your own names for describe and it.

For instance, suppose you're hunting a bug related to the database, and you want to pause execution after each example in a group to inspect the data. The Pry gem supports this style of debugging.[2] With Pry loaded, you can pause execution by calling binding.pry inside each it block.

It'd be easier and less error-prone, though, to have a method that acts like describe but adds the Pry behavior for you. Remember the x and f prefixes from Chapter 2, *From Writing Specs to Running Them*, on page 15? These let you skip or focus an example or group. Let's use the same technique to define new aliases, pdescribe and pit. Here's how you'd change one of your expense tracker examples from it to pit:

07-structuring-code-examples/02/expense_tracker/spec/integration/app/ledger_spec.rb

```
➤ pit 'successfully saves the expense in the DB' do
    result = ledger.record(expense)

    expect(result).to be_success
    expect(DB[:expenses].all).to match [a_hash_including(
      id: result.expense_id,
      payee: 'Starbucks',
      amount: 5.75,
      date: Date.iso8601('2017-06-10')
    )]
  end
```

To define both of these aliases, you'd add the following code to an RSpec.configure block:

07-structuring-code-examples/02/expense_tracker/spec/spec_helper.rb

```
RSpec.configure do |rspec|
  rspec.alias_example_group_to :pdescribe, pry: true
  rspec.alias_example_to :pit, pry: true
```

2. http://pryrepl.org